

Jupyter Notebook Execution Report

Name: Rodrigo Tapiador Cano

Date: April 04, 2026

Cell 1: ■ Markdown

Sesión 23: Taller de Forecasting con Modelos de Machine Learning

Objetivos:

- Aprender a transformar un problema de series temporales en un problema de regresión supervisada.
- Crear **características basadas en el tiempo** (ej. día de la semana, mes, año) para capturar la estacionalidad.
- Crear **características de retardo (lags)** para capturar la autocorrelación.
- Realizar una **división temporal** correcta de los datos (train/test).
- Entrenar y evaluar un modelo de ML (Random Forest, Gradient Boosting) para realizar predicciones.

Cell 2: ■ Markdown

1. Un Nuevo Enfoque para Predecir el Futuro

En lugar de usar modelos estadísticos especializados como ARIMA, hoy vamos a usar los potentes modelos de ML que ya conocemos. ¿Cómo? Reformulando la pregunta:

* **Pregunta Antigua:** ¿Cuál es la secuencia futura de la serie?

* **Pregunta Nueva:** Dado el día de la semana, el mes, el año y lo que pasó en los últimos días, ¿cuál será el valor **mañana**?

Esta nueva pregunta es un problema de **regresión estándar**. Solo necesitamos construir las características (X) y el objetivo (y).

Tu primera tarea: Carga nuestro dataset de consumo energético y déjalo listo con su `DatetimeIndex`.

Cell 3: ■ Code

```
# --- Imports y Carga del Dataset ---  
import pandas as pd
```

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

import lightgbm as lgb # Usaremos LightGBM, una implementación rápida de Gradient
Boosting

# Cargar los datos y prepararlos como en la sesión anterior
url = 'https://raw.githubusercontent.com/jenfly/opsd/master/opsd_germany_daily.csv'
df = pd.read_csv(url)
df['Date'] = pd.to_datetime(df['Date'])
df = df.set_index('Date')
df['Consumption'] = df['Consumption'].interpolate(method='linear')

print("Datos listos:")
print(df.head())

```

Output:

```

Datos listos:
              Consumption  Wind  Solar  Wind+Solar
Date
2006-01-01          1069.184   NaN   NaN         NaN
2006-01-02          1380.521   NaN   NaN         NaN
2006-01-03          1442.533   NaN   NaN         NaN
2006-01-04          1457.217   NaN   NaN         NaN
2006-01-05          1477.131   NaN   NaN         NaN

```

Cell 4: ■ Markdown

2. Creando Features del Calendario

La forma más sencilla de que un modelo entienda la estacionalidad es dándole características de calendario.

****Tu tarea:**** Crea 4 nuevas columnas en el DataFrame: `dayofweek`, `month`, `year` y `dayofyear`.

Cell 5: ■ Code

```
# --- Creando Features de Tiempo ---  
# TU CÓDIGO AQUÍ:  
# Extrae la información del índice. Pista: df.index.dayofweek  
  
df['dayofweek'] = # ...  
df['month'] = # ...  
df['year'] = # ...  
df['dayofyear'] = # ...  
  
print("DataFrame con nuevas características de tiempo:")  
print(df.head())
```

Error:

```
Traceback (most recent call last):  
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401  
    exec('\n'.join(lines[:-1]), glb)  
    ~~~~^  
  File "<string>", line 4  
    df['dayofweek'] = # ...  
                   ^^^^^  
SyntaxError: invalid syntax  
During handling of the above exception, another exception occurred:  
Traceback (most recent call last):  
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 408  
    exec(source, glb)  
    ~~~~^  
  File "<string>", line 4  
    df['dayofweek'] = # ...  
                   ^^^^^  
SyntaxError: invalid syntax
```

Cell 6: ■ Markdown

3. Creando Features del Pasado (Lags)

La información más valiosa para predecir el mañana es lo que pasó hoy y ayer. Crearemos "lags" o variables de retardo usando el método `.shift()`.

****Tu tarea:**** Crea 3 columnas de lag: el consumo del día anterior (`lag_1`), el de hace 2 días (`lag_2`) y el del mismo día de la semana pasada (`lag_7`).

Cell 7: ■ Code

```
# --- Creando Features de Retardo ---
```

```
# TU CÓDIGO AQUÍ:
# Pista: df['Consumption'].shift(1)
#df['lag_1'] = ...
#df['lag_2'] = ...
#df['lag_7'] = ...

# Como los shifts crean NaNs al principio, eliminamos esas filas
#df = df.dropna()

print("DataFrame con características de retardo:")
print(df.head())
```

Output:

```
DataFrame con características de retardo:
      Consumption  Wind  Solar  Wind+Solar
Date
2006-01-01      1069.184  NaN   NaN         NaN
2006-01-02      1380.521  NaN   NaN         NaN
2006-01-03      1442.533  NaN   NaN         NaN
2006-01-04      1457.217  NaN   NaN         NaN
2006-01-05      1477.131  NaN   NaN         NaN
```

Cell 8: ■ Markdown

4. La Regla de Oro: División Temporal

****¡MUY IMPORTANTE!**** En series temporales, NUNCA podemos usar `train\_test\_split` porque mezcla el pasado y el futuro. Debemos dividir los datos cronológicamente: entrenamos con el pasado y probamos con el futuro.

****Tu tarea:**** Divide los datos en un conjunto de entrenamiento (todo antes de 2017) y uno de prueba (todo desde 2017 en adelante).

Cell 9: ■ Code

```
# --- División Temporal (Train/Test Split) ---
# TU CÓDIGO AQUÍ:
#train = # ... Pista: df.loc['2006-01-01':'2016-12-31']
#test = # ... Pista: df.loc['2017-01-01:']
```

```
# Ahora separamos X e y para cada conjunto
#FEATURES = ['dayofweek', 'month', 'year', 'dayofyear', 'lag_1', 'lag_2', 'lag_7']
#TARGET = 'Consumption'

#X_train = train[FEATURES]
#y_train = train[TARGET]

#X_test = test[FEATURES]
#y_test = test[TARGET]
```

Cell 10: ■ Markdown

5. ¡A Entrenar!

Ahora viene la parte fácil. Como ya tenemos nuestro `X\_train`, `y\_train`, etc., podemos usar cualquier modelo de regresión que conozcamos.

****Tu tarea:**** Entrena un `RandomForestRegressor` y visualiza sus predicciones comparadas con los valores reales.

Cell 11: ■ Code

```
# --- Entrenamiento y Predicción ---
# TU CÓDIGO AQUÍ:

# 1. Crea una instancia de RandomForestRegressor (puedes usar n_estimators=100,
random_state=42)

# 2. Entrénalo con .fit(X_train, y_train)

# 3. Haz predicciones sobre el conjunto de test con .predict(X_test)

# 4. Calcula el RMSE (Root Mean Squared Error) para ver qué tan bueno es.

# Creando un DataFrame para visualizar
#predictions_df = pd.DataFrame({'real': y_test, 'prediccion': y_pred_rf})
#predictions_df.plot(figsize=(15, 8), title='Predicciones vs. Valores Reales')
#plt.show()
```

Cell 12: ■ Markdown

■ Reto para Casa: Prediciendo Pasajeros con Gradient Boosting

****Contexto:**** Ahora te toca a ti con el dataset "Air Passengers".

Tu Reto:

1. **Carga** los datos y crea las **features de tiempo** (``month`, `year``) y de **retardo** (prueba con ``lag_1`, `lag_2`` y ``lag_12`` para la estacionalidad anual).
2. Realiza una **división temporal**, usando los últimos 2 años para el conjunto de test.
3. Entrena un modelo **LightGBM** (``lgb.LGBMRegressor``) para predecir el número de pasajeros.
4. **Visualiza** las predicciones contra los valores reales.
5. **Analiza** la importancia de las características (``feature_importances_``).
6. **Responde a las siguientes preguntas de reflexión**:
 - * ¿Cuál es la regla más importante al dividir un dataset de series temporales y por qué?
 - * En la ingeniería de características, ¿qué tipo de patrón captura una 'feature de lag' y qué tipo de patrón captura una 'feature de tiempo' (como el mes)?
 - * ¿Qué características fueron las más importantes para tu modelo LightGBM? ¿Tiene sentido el resultado?

Cell 13: ■ Markdown

Proyecto Avanzado: Predicción del Precio de Cierre de las Acciones de Apple

Objetivo: Construir un modelo de Machine Learning para predecir el precio de cierre de las acciones de Apple (``Adj. Close``) basándose en su historial.

El Desafío: Los datos del mercado de valores son notoriamente "ruidosos" y difíciles de predecir. Nuestro objetivo es crear características de ingeniería (lags, medias móviles) que ayuden a un modelo potente como LightGBM a capturar la tendencia y la dinámica reciente del precio.

Cell 14: ■ Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import lightgbm as lgb
from sklearn.metrics import mean_squared_error

# URL directa de un repositorio de ejercicios de Pandas muy estable
url = 'https://raw.githubusercontent.com/guipsamora/pandas_exercises/master/09_Time_Series/Apple_Stock/appl_1980_2014.csv'

df = pd.read_csv(url)
```

```
# 1. Convierte 'Date' a datetime
df['Date'] = pd.to_datetime(df['Date'])

# 2. Establece 'Date' como índice
df = df.set_index('Date')

# 3. Ordena el índice
df.sort_index(inplace=True)

# CORRECCIÓN AQUÍ: Usamos 'Adj Close' sin el punto
apple_stock = df[['Adj Close']].copy()

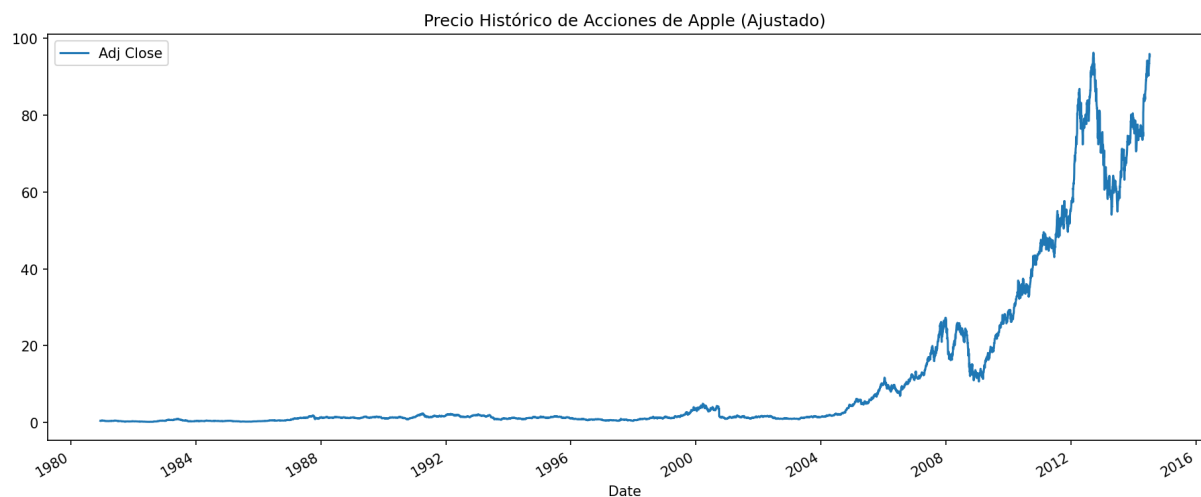
print("Dataset de Acciones de Apple:")
apple_stock.plot(figsize=(15, 6), title='Precio Histórico de Acciones de Apple (Ajustado)')
plt.show()
```

Output:

Dataset de Acciones de Apple:

[STDERR]

<string>;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 15: ■ Markdown

1. Ingeniería de Características para Datos Financieros

Para predecir el precio de mañana, el modelo necesita saber qué ha pasado recientemente.

****Tu Tarea:****

1. ****Crea características de calendario:**** `dayofweek`, `month`, `year`.
2. ****Crea características de retardo (Lags):**** El precio de cierre de los últimos 5 días (`lag\_1` a `lag\_5`).
3. ****Crea características de ventana móvil (Rolling):**** La ****media**** y la ****desviación estándar**** del precio en las últimas 2 semanas (14 días) y en los últimos 2 meses (60 días). Esto captura la tendencia y volatilidad a corto y medio plazo.

Cell 16: ■ Code

```
# --- Código
```

Cell 17: ■ Markdown

2. División Temporal y Modelado

Vamos a entrenar nuestro modelo con todos los datos hasta el final de 2012 y validaremos su rendimiento en el periodo 2013-2014.

****Tu Tarea:****

1. Divide los datos en entrenamiento y validación.
2. Define tus `FEATURES` y `TARGET`.
3. Entrena un `lgb.LGBMRegressor` y visualiza sus predicciones.
4. Analiza la importancia de las características.

Cell 18: ■ Code

```
# --- Código
```